Lab 04 — The Agent Team

One thing today: rebuild your desk as an agent team whose process is a graph in code, with a verifier no run can skip, and prove the verifier on a seeded error. (45 minutes, after the teaching.)

You arrive with: your desk from Lab 03 with every number sourced, your Costco and partner-pick reports on GitHub, your framework chosen, your one line on who decides which role runs next.

Open — the paired review (10 minutes)

1. **Tools alive.** Your desk runs on COST; your saved Costco report opens; your AI agent answers in your work folder.
2. **Defend.** Show your partner the source under one number in your report. Partner opens the filing and finds it, or does not, in under a minute.
3. **Gap check.** In your Lab 03 desk, what decides which role runs next: code or a model? Write one line. If you do not know, read your Manager's code until you do.

D — the question, the same for everyone

> Can the same four roles run as an agent team whose process is fixed in code, the same on every run, with a verifier that runs every time, and does that verifier catch a number the filings do not support?

R — what you work from

Input	Where
Your desk, its roles' instruction files and its saved runs	your repository

Input	Where
The paper's team, the thing to beat	Francis & Zhang (2026), Section 3.1
The graph and the verifier	Week 3 page, Teaching 1 and 2
A framework	one of the four the Week 3 page names; your choice goes in the bracket

I — build it from the instruction

1. Send your AI coding agent this request word for word, filling the bracket:

> Rebuild my desk as an agent team in Python with [LangGraph, the OpenAI Agents SDK, PydanticAI or CrewAI Flows]. Keep my seven steps as the team's tools, and keep my four roles' instructions, each in its own file in a folder named roles. The process is a graph in code, with these nodes in this order: the steps; the Research Analyst; the Financial Analyst; the Investment Advisor; the Strategy Manager; the Verifier. The order is the same on every run. A model never decides which node runs next. Each role node is one model call with the instructions from its file, and it receives only the state the graph gives it, as JSON: the task, the context and the expected output. The Manager checks that the five parts are present and consistent (Overview; Market Positioning and Growth Outlook; Valuation Insights; Future Developments and Strategic Recommendations; Investment Recommendation); if a part fails, the Manager sends that part back to its role one time only, then compiles the report. No role runs more than two times in one run. The Verifier runs after the Manager, on every run. No option turns it off. It receives only the compiled report, the saved filing texts and the saved facts, and it skips the Investment Advisor's thesis. For every other sourced number it checks, in code: a number sourced to an XBRL tag and period against the facts file, exactly; a number sourced to a line, that the line appears in that filing's text, allowing for spacing and formatting; a ratio, by recomputing it from its quoted inputs. Then, with the model, it checks that the line states that number for that period in that definition. A number with no source is unsupported, and so is a ratio with no definition where the filing offers two. The Verifier marks each number correct, wrong or unsupported, and corrects wrong and unsupported numbers from the filings, with the real line or tag beside each. Every run saves, in its own folder named by company, date and time and never overwriting an earlier run: the filing texts, the facts, each node's input and output, the report, and the verified report with the table of marks under the first

> line; the first line stays the filings line. The program takes a ticker, an optional --peers list, an optional --price number, and an optional --verify-only path to a saved run; with --verify-only it runs the Verifier on that run and nothing else. Print each node's name as it starts. Nothing in the code names a company. Read my key as API_KEY from .env. Tell me the exact command to install the packages, the exact command to run the agent team on a ticker with peers and a price, and the exact command to verify a saved run.

Rewrite it in ASD-STE100 yourself before you send it, as Week 1 taught; a sample is here.

2. Run the install command your agent gives you, then the run command: `COST --peers WMT,TGT,BJ --price P`, with the close your Lab 03 run used. *Expect:* the node names print in the graph's order, then the path of the verified report with its table of marks.
3. **While it runs:** open a second AI chat and work one item from your gap list. Never collected.
4. Run it again on COST. *Expect:* the same node order; a new run folder; the first run untouched.
5. Something not working? Debug with your AI: paste the exact command and the exact error text.

V — prove it on the known answer

1. **The seeded error.** Ask your agent to copy your saved Costco run and, in the copy's report, change the diluted EPS to a figure one dollar higher, with its source line changed to match. Verify the copy with `--verify-only`. *Expect:* that EPS marked wrong and corrected to the 10-K's figure, with the 10-K's per-share line beside it.
2. **The invented number.** In another copy, add a sentence that states revenue growth four percentage points above what the 10-K's two revenue figures give, with those two figures quoted. *Expect:* marked wrong or unsupported; the arithmetic from the quoted inputs beside it.
3. **The clean report.** Verify the run your Lab 03 audit marked correct on all eleven core claims (if your audit found an error, fix that number first). *Expect:* all eleven marked correct. A verifier that flags correct numbers is too eager, and that costs you as much as one that misses.

4. **The skip test.** Search the code for a way to run the agent team without the Verifier. *Expect:* none. If your agent added a flag that turns it off, have it removed.

Your partner checks the marks of test 1 against the 10-K, open at the line.

Orchestrator and evaluator — Learn on your own

1. What is it? Anthropic's *Building effective agents* names two workflows: orchestrator-workers (one call splits the task; workers do the parts) and evaluator-optimizer (one call produces; a separate call checks). Which is your Manager, and which is your Verifier?
2. How does it work here: what the Verifier receives, and what it never receives?
3. Explain it: why would letting the Verifier see the roles' instructions make it weaker?

Reflect

Explain to your partner: 1. one thing a model decided in your Lab 03 desk that code decides now; 2. one thing your verifier cannot catch, by design.

Before Session 5 — do these: the list on the Week 3 page, about 45 minutes.

Next: Lab 05 makes each role's work yours: a persona that must draw an insight, proved on a company your partner picks. Teams form; Checkpoint 2.